

Case Técnico — PM Pleno

Méliuz — In-App Browser

Entrega 1 — Análise do teste A/B/C e Recomendação

Maio de 2026

Sumário Executivo

Este documento analisa um teste A/B/C executado pela Méliuz em seu In-App Browser, comparando três experiências de saída para lojas parceiras durante 39 dias (11/dez/2025 a 19/jan/2026), com aproximadamente 1,2 milhão de visitas distribuídas entre as variantes.

Recomendação

Implementar a variante A (Controle). A variante B (saída via header) degrada conversão e receita de forma estatisticamente significativa (-9,3% RPV vs A), enquanto a variante C (saída via config) é estatisticamente equivalente a A em todas as métricas. Sem benefício mensurável e com maior complexidade técnica, C não justifica substituição do controle.

Principais achados

- B (Header): -2,02% em conversão e -9,33% em RPV vs A. Diferença comprovada estatisticamente ($p < 0,001$).
- C (Config): -1,09% em conversão e -2,10% em RPV vs A. Diferença não significativa ($p = 0,29$). Estatisticamente equivalente.
- Saídas externas convertem menos que saídas in-app em ambas variantes. O atrito do In-App Browser ajuda a segurar conversão.
- Saídas forçadas por login social têm o pior RPV de todas (R\$ 27-36). Apontam para uma fricção técnica que merece atenção.
- A parte INAPP da variante C é idêntica em performance à A — toda perda de C vem do tráfego direcionado a saídas externas.

Respostas às Perguntas do Case

1. Por que um app de cashback teria um In-App Browser?

Um In-App Browser é um navegador embarcado no app que abre conteúdo externo (como o site de uma loja parceira) sem que o usuário precise sair do ambiente do app. Para um app de cashback, isso resolve três problemas críticos:

- Atribuição confiável: garante que o tracking de cashback funcione, ligando a compra à visita originada no app. Em navegador externo, cookies, sessão e parâmetros podem se perder, comprometendo a atribuição da comissão.
- Controle da experiência: o app mantém visibilidade sobre o que o usuário faz, podendo medir cliques, tempo, abandonos e otimizar a jornada com base nesses sinais.
- Retenção no funil: ao manter o usuário dentro do app, reduz a chance de ele abandonar a jornada por distração (ex.: cair em outras abas do navegador, esquecer da compra).

Em troca, o In-App Browser sacrifica conveniências do usuário (login salvo, métodos de pagamento salvos, extensões), gerando um trade-off entre controle de produto e atrito de UX.

2. Qual problema de produto este teste parece tentar resolver?

O teste parece responder à hipótese de que o In-App Browser pode estar causando perda de conversão em casos específicos, particularmente quando o usuário precisa de funcionalidades que dependem do navegador do sistema — como login social (que não funciona dentro de WebViews por restrição técnica das plataformas Google, Apple e Facebook), uso de senhas salvas no autocomplete do Chrome/Safari, ou cartões salvos no navegador.

A presença consistente do `utm_term=LOGIN` nas variantes B e C confirma essa suspeita: parte das saídas externas são forçadas por necessidade técnica de login social. Ou seja, o produto está testando se oferecer uma alternativa de "escape" do In-App Browser (via header ou config) compensaria essa fricção e aumentaria a conversão.

3. Qual trade-off existe entre manter o usuário no In-App Browser e permitir saída para navegador externo?

Manter no In-App Browser	Permitir saída para navegador externo
✓ Tracking confiável de atribuição ✓ Controle da experiência ✗ Sem login social ✗ Sem cartões/senhas salvas	✓ Login social funciona ✓ Conveniências do navegador do usuário ✗ Risco de perder atribuição ✗ Usuário pode abandonar funil

O dado importante é que esse trade-off não é teórico — os resultados do teste mostram que, no caso da Méliuz, manter o usuário dentro do app gera mais receita. As conveniências do navegador externo não compensam a perda de controle.

4. Qual hipótese cada variante parece testar?

- Variante A (Controle): nenhuma hipótese — é o estado atual, ponto de referência.
- Variante B (Header): hipótese de que dar destaque visível à opção de sair (botão no topo) aumentaria a conversão de usuários que preferem o navegador externo. Testa o caso "preferência do usuário".
- Variante C (Config): hipótese de que oferecer a saída como opção avançada (escondida no menu) atenderia usuários que realmente precisam, sem expor a maioria a essa fricção. Testa o caso "opção avançada para usuários intencionados".

5. Como definir e calcular a métrica de sucesso?

A métrica primária de sucesso deve ser Receita por Visita (RPV), calculada como:

$$RPV = \text{Soma de sale_amount} / \text{Total de visitas}$$

Justificativa: RPV captura tanto conversão quanto ticket médio em uma única métrica, e reflete diretamente o valor gerado pelo produto. Métricas isoladas (apenas conversão ou apenas ticket) podem ser enganosas — uma variante pode converter mais com ticket menor (neutralizando o ganho) ou converter menos com ticket maior (compensando).

Métricas secundárias a acompanhar:

- Taxa de Conversão = visit_ids com pelo menos uma compra / visit_ids totais
- Ticket Médio = sale_amount / número de compras
- Comissão por Visita = expected_commission / visitas (a métrica mais próxima do impacto financeiro real para a Méliuz)

Importante: o grão de cálculo precisa ser tratado corretamente. Como uma visit_id pode ter múltiplas transações (até 306 em um caso extremo), a conversão deve ser calculada como flag binária por visit_id, não como contagem de transactions. A receita, por outro lado, deve ser somada agregadamente por visit_id.

6. Qual versão deve ser implementada? Por quê?

Resposta: Variante A (Controle).

Justificativa em três pontos:

1. Performance: A é estatisticamente equivalente ou ligeiramente superior a C, e claramente superior a B. Não há evidência de que C entregue mais receita ou conversão.
2. Custo de oportunidade: implementar C exige desenvolver e manter um novo componente (botão no menu config, fluxo de saída para navegador externo, instrumentação adicional). Esse esforço não retorna ganho mensurável.
3. Simplicidade operacional: princípio da parcimônia. Quando duas alternativas têm performance equivalente, escolhe-se a mais simples — A já está em produção, é conhecida pelo time, é estável.

Observação importante: a decisão de manter A não significa que o problema original (saídas forçadas por login social) está resolvido. Apenas significa que as soluções B e C propostas não foram eficazes. A próxima melhoria deve atacar a raiz do problema (login social dentro do In-App Browser), não oferecer rotas de fuga.

Análise Detalhada dos Dados

Estrutura e tratamento dos dados

Os dados foram entregues em 6 CSVs relacionais com diferentes grãos:

- visits.csv (1.197.465 linhas): grão = uma saída/click. Tabela central da análise.
- visit_url_metadata.csv (1.197.465 linhas): grão = uma saída. JSON com parâmetros customizados.
- url_params.csv (56.506 linhas): grão = um conjunto de UTMs (deduplicado).
- transactions.csv (160.269 linhas): grão = uma compra. Pode haver múltiplas compras por visita.
- partners.csv (435 linhas): dimensão de lojas parceiras.
- channels.csv (2 linhas): dimensão de canal final (INAPP ou BROWSERDEFAULT).

Identificação das variantes

A variante (A/B/C) não está em coluna direta — está dentro de um JSON na coluna tracking_url_params da tabela visit_url_metadata, no campo mz_test_gotoexternalbrowser. Foi necessário parsear o JSON para extrair essa informação.

Distribuição final após parse:

- A: 377.716 visitas (31,5%)
- B: 385.757 visitas (32,2%)
- C: 383.870 visitas (32,1%)
- Sem variante atribuída (null): 50.122 visitas (4,2%) — excluídas da análise

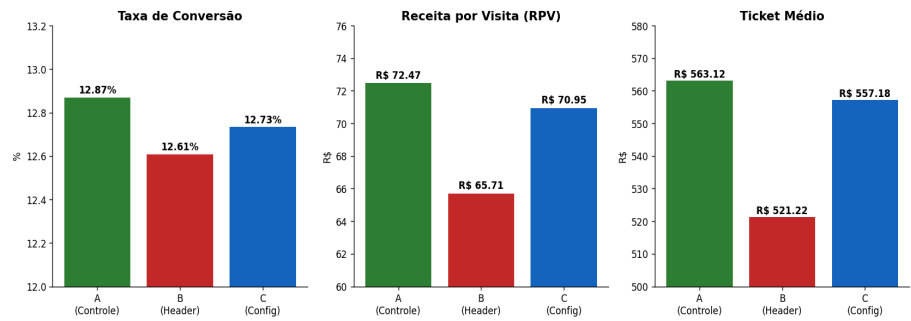
A distribuição entre A/B/C é equilibrada (diferença máxima de 2,1% entre maior e menor), indicando que a randomização do teste foi bem executada.

Validações de integridade realizadas

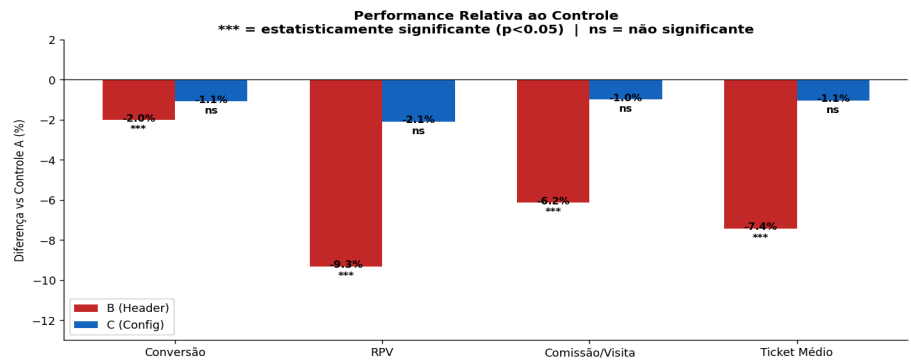
- Receita agregada por visit_id bate exatamente com soma direta de transactions (R\$ 84.395.596,91 = R\$ 84.395.596,91).
- Contagem de visitas convertidas (flag binária) bate com visit_ids únicos em transactions (153.531 = 153.531).
- Zero transações órfãs: todas as compras têm visita correspondente.
- visit_id é única em visits.csv e em visit_url_metadata.csv (não duplica no JOIN).

Resultado consolidado: A vs B vs C

Comparação das Variantes — Métricas Principais



Variante	Visitas	Conversão	RPV (R\$)	Ticket (R\$)
A (Controle)	377.716	12,87%	72,47	563,12
B (Header)	385.757	12,61%	65,71	521,22
C (Config)	383.870	12,73%	70,95	557,18



Teste de significância estatística

Para evitar a interpretação ingênua de pequenas diferenças, apliquei testes estatísticos para verificar se as diferenças observadas são reais ou ruído aleatório.

- Para conversão (variável binária sim/não comprou): z-test de duas proporções.
- Para receita por visita (variável contínua): t-test de Welch.
- Nível de significância: $p < 0,05$ (5% de chance de erro tipo I).

Comparação	Métrica	p-valor	Conclusão
A vs B	Conversão	0,0006	Diferença real
A vs B	RPV	< 0,0001	Diferença real
A vs C	Conversão	0,0763	Não significativa
A vs C	RPV	0,2916	Não significativa

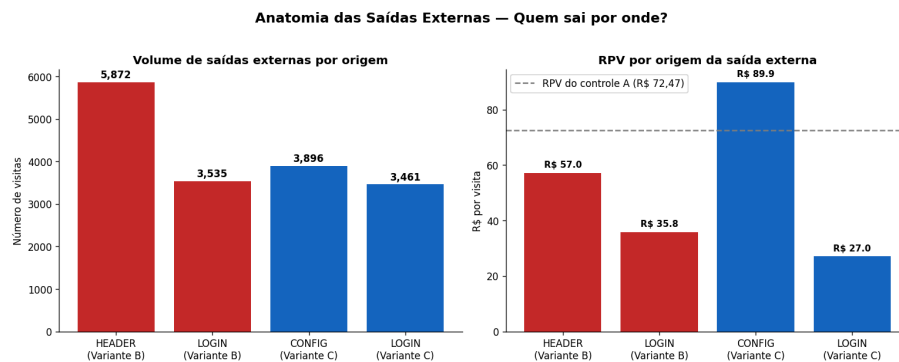
Conclusão dos testes:

4. B é comprovadamente pior que A em conversão e RPV ($p < 0,001$ em ambos). Não há dúvida estatística.
5. C é estatisticamente indistinguível de A. A pequena diferença observada (-2% em RPV) tem 29% de chance de ser ruído. Não posso afirmar que C é pior, mas também não posso afirmar que é melhor.

Anatomia das saídas externas

Toda a perda de performance das variantes B e C vem das saídas externas (utm_content = EXTERNAL_BROWSER_MODAL). Essas saídas têm três origens distintas, identificadas pelo utm_term:

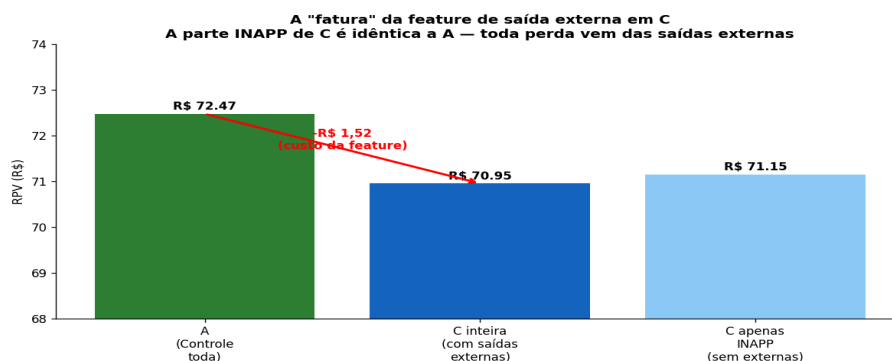
- **HEADER:** usuário clicou no botão visível no topo da tela. Existe apenas em B. Saída voluntária e fácil.
- **CONFIG:** usuário foi até o menu de configurações para acionar a saída. Existe apenas em C. Saída voluntária mas exige esforço.
- **LOGIN:** saída forçada quando o usuário tenta login social na loja (limitação técnica do In-App Browser). Existe em B e C.



Insights desta análise:

6. LOGIN tem o pior RPV de todas as saídas (R\$ 27-36). É uma saída forçada por fricção técnica, e o usuário tende a abandonar no caminho.
7. CONFIG tem o melhor RPV das saídas externas (R\$ 89,92, acima até do RPV do controle). Sugere auto-seleção: quem se dá ao trabalho de buscar a opção no menu é um usuário muito intencional.
8. HEADER converte e gera RPV pior que CONFIG, com volume muito maior. É a combinação que explica a queda agregada de B: muitas saídas + saídas de baixa qualidade.

Decomposição: A perda de C vem 100% das saídas externas

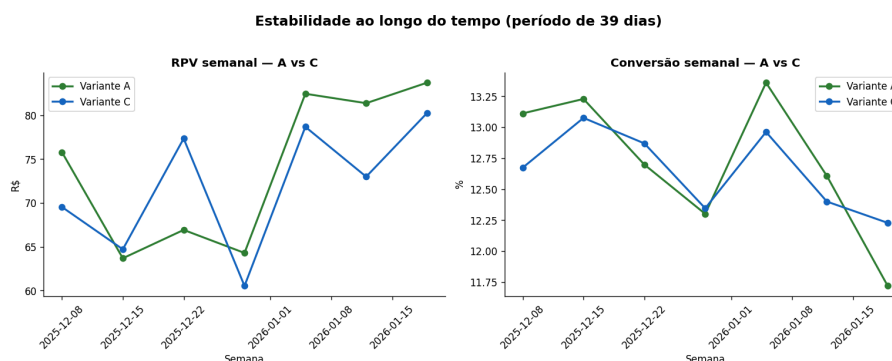


Quando isolamos as visitas com `utm_content = PARTNER_PAGE` na variante C (excluindo as saídas externas), o RPV é R\$ 71,15 — praticamente idêntico ao controle A (R\$ 72,47).

Isso confirma que a feature de saída externa em C não estraga a experiência padrão dos usuários que não a utilizam. Toda a queda de R\$ 1,52 em RPV vem exclusivamente das ~7.357 visitas que passaram pelo modal de saída externa.

Implicação: o problema de C não é a feature em si, mas o fato de que oferecer essa opção captura usuários que terão pior performance no navegador externo do que teriam tido no In-App Browser.

Estabilidade ao longo do tempo



Análise semanal mostra que A e C oscilam dentro de uma faixa similar, sem tendência sustentada de uma variante superando a outra. A é melhor em 5 das 7 semanas, mas as diferenças são pequenas e inconsistentes — coerente com a conclusão de que as duas variantes são estatisticamente equivalentes.

Insights Adicionais

LOGIN é uma dor invisível em A

O `utm_term=LOGIN` aparece apenas em B e C, com cerca de 3.500 visitas em cada. Isso não significa que o problema não exista em A — significa que A não tem instrumentação para esse caso.

Em A, quando o usuário tenta login social dentro do In-App Browser, provavelmente acontece um de dois cenários: o login falha por limitação técnica e o usuário abandona, ou o login funciona via algum workaround sem gerar evento de saída. Em qualquer dos casos, a fricção fica invisível nos dados.

Portanto, a comparação direta de saídas externas entre A e B/C subestima o problema real de login social. A próxima iteração de produto deveria buscar resolver login social dentro do In-App Browser, em vez de oferecer rotas de fuga.

Auto-seleção de usuários de alto valor via Config

Apesar do volume baixo, as saídas via CONFIG geraram RPV de R\$ 89,92 — o maior de qualquer segmento analisado, inclusive maior que o RPV de qualquer variante completa.

Isso sugere que usuários que buscam ativamente uma opção avançada no menu são usuários com alta intenção de compra. Vale uma investigação mais aprofundada: quem são esses usuários? São compradores frequentes? Em quais lojas eles convertem? Há um padrão por categoria?

Se for confirmada uma segmentação de "power users" via esse padrão, pode haver oportunidade de produto específica para esse público.

Sazonalidade do período

O teste cobriu um período sensível (Black Friday, Natal, Ano Novo). Vale considerar que comportamentos de compra nesse período podem diferir do baseline anual: maior pressa de compra, mais sensibilidade a fricção, possivelmente menor tolerância a navegadores secundários.

Recomendo, antes da implementação definitiva, validar se os resultados se mantêm em período não-sazonal (ex.: fevereiro a abril).

Perguntas que faria com mais tempo ou dados

- Quais lojas têm maior incidência de saída via LOGIN? Há um padrão por categoria de loja (marketplaces vs e-commerces verticais)?
- O comportamento muda por sistema operacional (iOS vs Android)? As limitações de WebView são diferentes.
- Quais `customer_ids` tiveram saídas via CONFIG? São usuários recorrentes? Qual o LTV deles?
- Qual o impacto secundário das saídas externas em retenção (o usuário que sai pro navegador externo volta ao app no futuro com mesma frequência)?
- Há diferença entre primeira visita e visitas recorrentes? Talvez B funcione melhor para usuários novos que ainda não conhecem o In-App Browser.

Riscos, Limitações e Cuidados

Limitações da análise

- Período sazonal: 39 dias cobrindo Natal/Ano Novo. Resultados podem não generalizar para o ano inteiro.
- 4,2% das visitas (50.122) ficaram sem variante atribuída e foram excluídas. Vale investigar o motivo da ausência de tag.
- Não temos dado de plataforma (iOS/Android), modelo de aparelho, versão do app — variáveis que poderiam moderar o efeito do teste.
- A análise considera apenas eventos de saída e compra. Não vemos comportamento intermediário (tempo no In-App, scrolls, abandono parcial).

Riscos da recomendação (manter A)

- O problema de login social permanece não resolvido. A fricção continua existindo, apenas fica invisível.
- Usuários que efetivamente preferem o navegador externo não terão essa opção. Pequeno custo de UX para uma minoria.
- Há risco competitivo se concorrentes oferecerem navegação mais flexível — vale monitorar feedback qualitativo.

Riscos de implementar B ou C apesar dos dados

- B: perda comprovada de receita. Se for implementada, a Méliuz perderia receita real e mensurável.
- C: investimento de desenvolvimento sem retorno mensurável. Cria complexidade no código sem ganho.

Metodologia e Processo de Análise

Ferramentas utilizadas

- Análise de dados: Python (pandas, numpy, scipy) para processamento e testes estatísticos.
- Visualização: matplotlib para gráficos.
- IA assistente: Claude (Anthropic) para co-análise, validação de raciocínio e estruturação dos entregáveis.

Etapas executadas

9. Inspeção inicial dos arquivos: tamanho, número de linhas, esquema de cada CSV.
10. Sanity check: verificação de unicidade de chaves, integridade referencial, distribuição de valores.
11. Construção da tabela mestre: JOIN das 6 tabelas em um dataframe único, tratando corretamente o grão (uma linha por visita, com flag binária de conversão e receita agregada).
12. Parse do JSON: extração das variantes A/B/C do campo tracking_url_params.
13. Análise comparativa: cálculo de métricas por variante (conversão, RPV, ticket médio, comissão por visita).
14. Testes de significância: z-test e t-test para validar se as diferenças observadas são reais.
15. Análise das saídas externas: desagregação por utm_content e utm_term.
16. Análise por parceiro e temporal: validação de que a conclusão não muda em segmentações.
17. Síntese e recomendação.

Validações para reduzir risco de erro

- Confronto de totais agregados com totais diretos das tabelas originais (receita e contagens).
- Verificação de unicidade de chaves antes de cada JOIN.
- Cálculo de conversão como flag binária por visit_id (evitando duplicação por múltiplas transações).
- Uso de testes estatísticos para evitar conclusões baseadas em ruído.
- Análise por subgrupo (top parceiros, semanas) para validar robustez da conclusão agregada.